Zigで覗く、Linuxの非同期I/O

poll・epoll・io_uring を、動かして見る

いつも使う async/await の、少し下へ。

async/awaitの下では何が起きている？

レスポンスを待つ間、
「データが届いた」と誰が教えてくれる？



典型的なネットワークI/Oの一例。言語・runtime・操作によって経路は異なる。

その下にあるLinuxの仕組みを、直接見てみる

Zigとは？

Cに近い低レイヤーを扱える
システムプログラミング言語

メモリやリソースを明示的に管理する

今回は std.os.linux

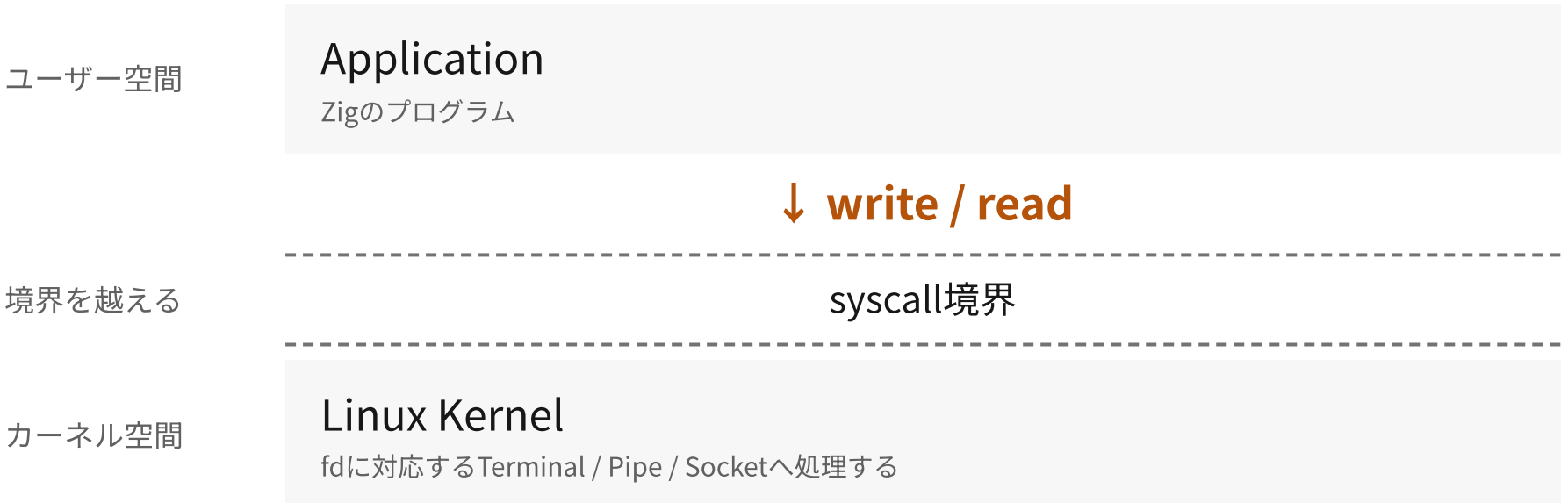
libcを経由せず、Linuxの
システムコールを呼び出す

ミニデモ：write

write → **hello** → **戻り値**

ターミナルへ ↗

システムコールは、OSへの依頼



普通の関数呼び出しの先で、カーネルに処理を頼む

fdは、プロセスが持つ番号

0 / 1 / 2

stdin / stdout / stderr

そのほかの番号

ファイル / pipe / socket …

プロセスごとのfdテーブルで管理される。

pipe：データの通り道

書き込み側の**fd**

↓ **hello**

読み取り側の**fd**

ミニデモ：blocking read

データなし　→ readが待つ →　**到着後にhello**

ターミナルへ ↗

pollは、fdがReadyになるのを待つ

Application	→	poll	→	Kernel
監視するfd一覧		毎回、一覧を渡す		ReadyなfdをReturn

non-blocking read
まだデータがない → EAGAIN

pollで待つ
読めるfdが分かる → readする

Readyは「読み取りを試せる状態」。データはreadで取る

2本のpipe、届くのは片方だけ

01 空のpipeでは **EAGAIN**

02 **poll** が返すのは **Readyなfd**

03 そのあと **read** すると **hello**

ターミナルへ ↗

ソース → 実行 → strace

epollは、監視対象を登録しておく

Application	→	epoll_ctl	→	Kernel
fd A / fd B		監視集合を登録		監視集合を保持

epoll_wait → Readyなfd → read → hello

一覧を毎回渡すpollと、登録を保持するepoll

同じpipe、同じhelloで見てみる

01 **epoll_ctl ADD** で **2つのfd**を登録

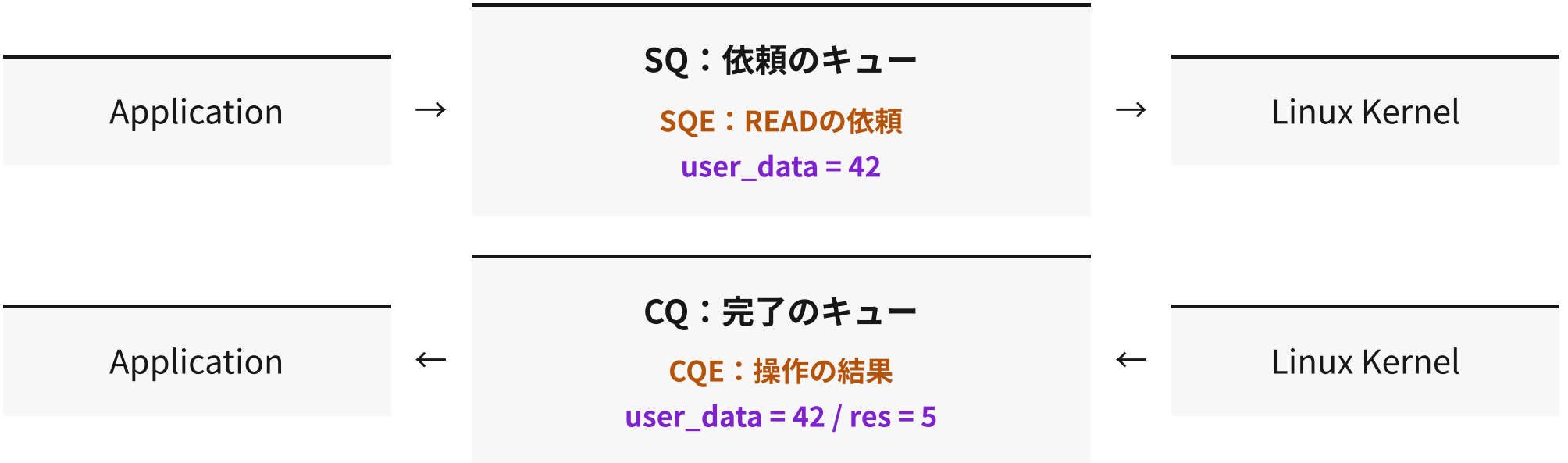
02 **epoll_wait** が **Readyなfd**を返す

03 **read** で **hello**を取得

ターミナルへ ↗

登録と待機を分けて見る

io_uringは、操作を頼んで完了を受け取る



今回はoneshot READ。依頼1件と、その完了を対応付ける

READを投入すると、何が返る？

01 **SQE** に **pipeのfd** と **42** を設定

02 **submit** して、データの到着を待つ

03 **CQE** は **42 / 5 byte**、bufferには **hello**

ターミナルへ ↗

SQE → submit → CQE → buffer

「読み取れる」と「読み取った」の違い

poll / epoll：Readiness

読める状態のfdが分かる

↓

アプリが**read**する

↓

helloを取得

io_uring READ：Completion

READ操作を投入する

↓

カーネルが読み取りを処理する

↓

完了結果と**hello**を取得

通知されるのは、状態か、操作の完了か

短いプログラムでも、役割が見える

01 **fd**は、カーネルへの依頼で使う番号

02 **poll / epoll**は、複数のI/Oを待つ道具

03 **io_uring**は、操作と完了をキューでやり取りする

straceで見えるのはsyscallの境界。共有メモリ内の操作までは見えない。

非同期処理の、少し下を覗いてみた

01 async/await の下には、OSのI/O機構がある

02 Ready通知とCompletion通知では、依頼の仕方が違う

03 Zigとstraceで、その動きを直接確かめられる

普段何気なく使う非同期処理。
その少し下に、Linuxの面白い仕組みがある。

デモと参考資料

短いZigプログラム

- github.com/moribit/zig-syscall-lt / demos
- Zig 0.17.0 Documentation

非同期処理の実装例

- libuv：Design overview
- Python：Event Loop

Linux man-pages

- syscalls(2) / pipe(7)
- poll(2)
- epoll(7)
- io_uring(7)

過去のベンチマーク・分析・旧スライドはリポジトリ内に保存。